Electronics Design with VHDL.

Objects, data types and operators (V.0)

Dr. Carlos Cruz de la Torre

Department of Electronic. Área: Tecnología Electrónica

University of Alcalá.

01/09/2024

1: Data Type Conversion
Given the signal declaration signal x : std_logic_vector(3 down to 0) := "1010";
Indicate to statement to correctly convert x into an integer

2: Logical Operators
Determine which of the following sentences is a valid logical operation in VHDL
A. signal a : std_logic; a := a AND '1';
B. signal b : std_logic; b <= b OR '0';
C. signal c : std_logic_vector(3 downto 0); c <= c XOR "1010";
D. signal d : std_logic; d := d NAND '1';

3: Aggregate Allocation
Determine a valid way to assign all bits of a 4-bit vec std_logic_vector signal to '0'

4: Relational Operators
Given two signals a and b of type std_logic_vector(3 to 0), determine an expression that checks whether a is less than b?

5: Matrix Statement
Declare a std_logic_vector array1 array with 8 elements, each 4 bits wide

6: Basic Data Types
What will happen when you compile the following VHDL code?

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity BasicDataTypesTest is
end BasicDataTypesTest;

architecture Behavioral of BasicDataTypesTest is
   signal a : bit := '1';
   signal b : std_logic := '0';
begin
   process
   begin
      a <= b;
      wait;
   end process;
end Behavioral;
```

7: Arrays and Vectors
Determine the result of the following VHDL code after simulation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ArrayTest is
end ArrayTest;

architecture Behavioral of ArrayTest is
   signal arr : std_logic_vector(3 downto 0) := "1010";

begin
   process
      variable temp : std_logic_vector(3 downto 0);
   begin
      temp := arr(1 downto 0) & arr(3 downto 2);
      wait;
   end process;
end Behavioral;
```

8: Record Types
Determine the incorrect assignments following the next VHDL code

```
type person is record
      name : std_logic_vector(7 downto 0);
      age  : integer range 0 to 127;
end record;
      signal john : person := (name => "John", age => 25);

      A. john.name <= "01001010";
      B. john.age <= 30;
      C. john.age <= -1;
      D. john.name <= (others => '0');
```

9: Signal and Variable Behaviour
Determine the value of output_signal after the process executes of the following VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SignalVarTest is
   port (
      clk : in std_logic;
      output_signal : out std_logic
   );
end SignalVarTest;
```

```
architecture Behavioral of SignalVarTest is
  signal signal_var : std_logic := '0';
begin
  process(clk)
    variable temp_var : std_logic := '0';
  begin
    if rising_edge(clk) then
      temp_var := not temp_var;
      signal_var <= temp_var;
    end if;
    output_signal <= signal_var;
  end process;
end Behavioral;
```

A. output_signal toggles between '0' and '1' on every rising edge of the clock.
B. output_signal always remains '0'.
C. output_signal toggles between '1' and '0' but with one clock cycle delay.
D. output_signal will be undefined.

10: Data Type Compatibility
Determine the result of the following VHDL code after simulation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TypeTest is
end TypeTest;

architecture Behavioral of TypeTest is
  signal std_logic_var : std_logic := '1';
  signal bit_vector_var : bit_vector(3 downto 0) := "1010";
begin
  process
  begin
    std_logic_var <= bit_vector_var(0);
    wait;
  end process;
end Behavioral;
```

A. The code will compile and simulate successfully, assigning '0' to std_logic_var.
B. The code will compile, but a runtime error will occur due to type mismatch.
C. The code will not compile due to a type mismatch error.
D. The code will compile and assign '1' to std_logic_var.

11: Arithmetic Operations with Different Data Types
Determine the result of the following VHDL code, and the result_signal after the process executes

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
entity ArithmeticTest is
end ArithmeticTest;

architecture Behavioral of ArithmeticTest is
   signal a : std_logic_vector(3 downto 0) := "0110"; -- 6 in decimal
   signal b : std_logic_vector(3 downto 0) := "0011"; -- 3 in decimal
   signal result_signal : std_logic_vector(4 downto 0);
begin
   process
   begin
      result_signal <= ('0' & a) + ('0' & b);
      wait;
   end process;
end Behavioral;
```

A. result_signal will hold "10001".
B. result_signal will hold "00001".
C. result_signal will hold "01001".
D. The code will not compile due to an operator error.

12: Logical Operators on std_logic_vector
Determine the result of the following VHDL code, and the value of result_vector

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity LogicalOperationTest is
end LogicalOperationTest;

architecture Behavioral of LogicalOperationTest is
   signal vec1 : std_logic_vector(3 downto 0) := "1101";
   signal vec2 : std_logic_vector(3 downto 0) := "1011";
   signal result_vector : std_logic_vector(3 downto 0);
begin
   process
   begin
      result_vector <= (vec1 and vec2) or (vec1 nand vec2);
      wait;
   end process;
end Behavioral;
```

13: Aggregate Assignments
Determine the result of the following VHDL code (assume output_vector is defined as a 4-bit vector)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity AggregateAssignmentTest is
end AggregateAssignmentTest;
```

```
architecture Behavioral of AggregateAssignmentTest is
   signal input_vector : std_logic_vector(3 downto 0) := "1101";
   signal output_vector : std_logic_vector(3 downto 0);
begin
   process
   begin
      output_vector <= (others => input_vector(0));
   end process;
end Behavioral;
```

14: What is the difference between std_logic_vector type and signed and unsigned types?

15: Determine library and package that should we include in a VHDL design if we want to use signed and unsigned types

16: Determine the difference between whole types and signed and unsigned types

17: What is the difference between the whole type and the positive type?

18: Write VHDL code that converts 8 bit signed signal to a std_logic_vector,
Perform the conversion of the resultant std_logic_vector to an unsigned type.

19: Write a VHDL code that converts an integer type to an 8 bit std_logic_vector. The integer should be treated as an unsigned number.

20: Create a VHDL entity and architecture that defines a signal sig1 of type std_logic. Assign a constant value '1' to this signal

21: Create a VHDL entity that defines an std_logic_vector of length 4. Use logical operators to perform a bitwise AND operation between two vectors.

22: Create a VHDL entity that defines an array of type std_logic_vector. Write a process to reverse the bits of the vector using a loop.

23: Create a VHDL entity that performs an arithmetic addition of two 4-bit unsigned numbers and stores the result in a 5-bit vector to handle overflow.

24: Create a VHDL entity that compares two 4-bit vectors using relational operators (<, >, =). Set a signal is_greater to '1' if the first vector is greater than the second.

25: Attributes

1. signal data : std_logic_vector(7 downto 0), what does the data'range attribute return?
2. if (clk'event and clk = '1') then
3. signal vector_signal : std_logic_vector(15 downto 0); What does the vector_signal'length attribute return?
4. signal b : std_logic_vector(3 to 10); What value will the b'high attribute return?
5. What do you get by using the 'delayed(5 ns) attribute on a signal?
6. Define an attribute called my_attribute for a signal and assign it an integer value of 10.

Electronics Design with VHDL.

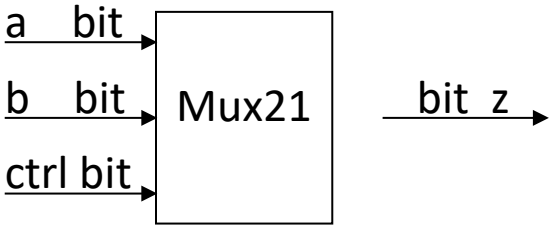
BASIC UNIT (V.0)

Department of Electronic. Área: Tecnología Electrónica

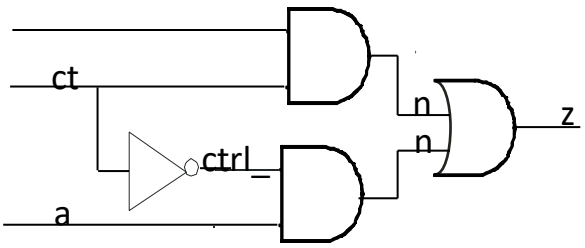
University of Alcalá.

01/09/2024

1: Architecture of Mux21 in algorithmic style



2: Architecture Mux21 in dataflow style



3: Structural architecture of Mux21

4: Given the following description in VHDL, determine the coding style

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity adder is
   Port ( A : in  STD_LOGIC_VECTOR(3 downto 0);
          B : in  STD_LOGIC_VECTOR(3 downto 0);
          SUM : out  STD_LOGIC_VECTOR(4 downto 0));
end adder;

architecture behavioral of adder is
begin
   process(A, B)
   begin
      SUM <= ("0" & A) + ("0" & B); -- Concatenamos un bit para evitar overflow
   end process;
end behavioral;
```

5: Determine the type of description used in the following VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity and_gate is
   Port ( A : in  STD_LOGIC;
          B : in  STD_LOGIC;
          Y : out  STD_LOGIC);
end and_gate;
architecture dataflow of and_gate is
begin
   Y <= A AND B;
end dataflow;
```

6: Design a 2-bit comparator that determines whether two 2-bit numbers are equal, less, or larger using Algorithmic style.

7: Design a 4-bit adder that calculates the sum of two 4-bit numbers using the Dataflow style.

8: Design a 4-to-1 multiplexer using basic logic gates, in structural style.

9: Implements a Flip-Flop type D sensitive to the watch's rise edge, with asynchronous reset input using Algorithmic style**.**

10: Implement a 2-to-4 decoder using Dataflow style.

11: Design a two-input OR circuit (A, B) using only NAND gates in structural style. Use the logical expression of OR using NAND:A+B=((A·A)·(B·B))A+B=((A·A)·(B·B))

12: Implement a 1-bit Full Adder using two Half Adders and an OR gate. Half Adders must add two input bits and provide one addition bit and one carry bit. The Full Adder adds three bits (A, B, and Carry-in) and generates a sum and a carry.

13: Design a 4-bit binary counter that increases its value on each rising edge of the clock. When you reach the maximum value (1111), you will need to reset to 0000.

14: Design a 4-bit adder that takes two 4-bit inputs (A and B) and returns a summed 4-bit output (Sum) and a carry bit (Carry).

Simulation models

15: Create a VHDL design that implements basic logic gates like AND, OR, and NOT. Design a testbench to simulate the behavior.

16: Design a simple half-adder adds two single bits and produces a sum and carry output.

17: Design a sequential logic element, the D flip-flop, captures the input on a rising clock edge.

18: Design a simple 4-bit counter that increments on each rising clock edge.